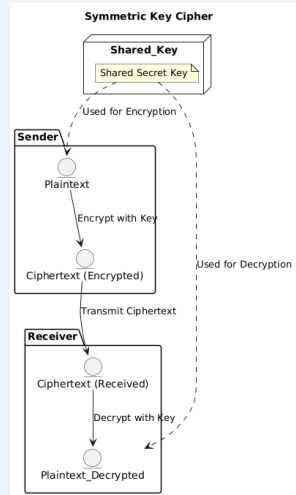


Cryptography: Symmetric and Asymmetric Key Ciphers

Unit III

- 1 The sender uses a shared secret key to encrypt the plaintext into ciphertext.
- 2 The receiver uses the same secret key to decrypt the ciphertext back into plaintext.
- 3 Both parties must securely exchange the secret key beforehand.

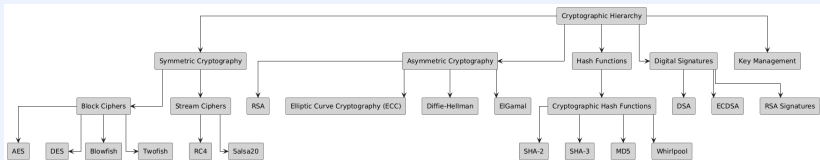


- **Block Ciphers**

- Encrypts data in fixed-size blocks (e.g., 64-bit, 128-bit).
- Examples: AES, DES.

- **Stream Ciphers**

- Encrypts data one bit or byte at a time.
- Examples: RC4, Salsa20.



- The Data Encryption Standard (DES) is a symmetric key block cipher.
- Developed by IBM in the 1970s and adopted as a standard by NIST in 1977.
- Operates on 64-bit blocks of data using a 56-bit key.
- Widely used for securing data but has been largely replaced by stronger algorithms (e.g., AES).

-
- ```

graph TD
 Input[Key with parity bits
(64 bits)] --> ParityDrop[/Parity drop/]
 ParityDrop --> CipherKey[Cipher key (56 bits)]
 CipherKey -- 28 bits --> Shift1L[Shift left]
 CipherKey -- 28 bits --> Shift1R[Shift left]
 Shift1L -- 28 bits --> Comp1[/Compression P-box/]
 Shift1R -- 28 bits --> Comp1
 Comp1 -- 28 bits --> Shift2L[Shift left]
 Comp1 -- 28 bits --> Shift2R[Shift left]
 Shift2L -- 48 bits --> RoundKey1[Round Key 1]
 Shift2R -- 48 bits --> RoundKey1
 RoundKey1 -- 48 bits --> Shift3L[Shift left]
 RoundKey1 -- 48 bits --> Shift3R[Shift left]
 Shift3L -- 28 bits --> Comp2[/Compression P-box/]
 Shift3R -- 28 bits --> Comp2
 Comp2 -- 28 bits --> Shift4L[Shift left]
 Comp2 -- 28 bits --> Shift4R[Shift left]
 Shift4L -- 48 bits --> RoundKey2[Round Key 2]
 Shift4R -- 48 bits --> RoundKey2
 RoundKey2 -- 48 bits --> Shift5L[Shift left]
 RoundKey2 -- 48 bits --> Shift5R[Shift left]
 Shift5L -- 28 bits --> Comp3[/Compression P-box/]
 Shift5R -- 28 bits --> Comp3
 Comp3 -- 28 bits --> Shift6L[Shift left]
 Comp3 -- 28 bits --> Shift6R[Shift left]
 Shift6L -- 48 bits --> RoundKey16[Round Key 16]
 Shift6R -- 48 bits --> RoundKey16

```

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- In DES, the initial 64-bit key includes 8 parity bits (1 for each byte).
- The parity bits are dropped, reducing the key size to 56 bits.
- A predefined table determines the positions of the retained 56 bits.

|               |   |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|---|
| Byte          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| Original Bits | 8 | 8 | 8 | 8 | 8 | 8 | 8 | 8 |
| Retained Bits | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 |





## Overview of Left Shift Operation

- The 56-bit key (after Parity Drop) is split into two 28-bit halves:
  - *C*: Left half
  - *D*: Right half
- Each half undergoes **left circular shifts** (1 or 2 bits) in each round.
- Shifting introduces variability in subkeys for each of the 16 rounds.
- Shift schedule:
  - Shift by 1 bit in rounds: 1, 2, 9, 16.
  - Shift by 2 bits in other rounds.

## Example Key (Initial Values)

Initial 56-bit key (after Parity Drop):

$$C_0 = 1010101010111010101010111010 \quad (\text{Left half})$$

$$D_0 = 1100110011001100110011001100 \quad (\text{Right half})$$

- We will perform left circular shifts on  $C$  and  $D$  for the first few rounds.
- The results will be used to generate round-specific subkeys.

## Round 1 and Round 2 Shifts

Initial 56-bit key (after Parity Drop):

$$C_0 = 1010101010111010101010111010 \quad (\text{Left half})$$

$$D_0 = 1100110011001100110011001100 \quad (\text{Right half})$$

### Round 1: Left Shift by 1 Bit

$$C_1 = \text{Shift}(C_0, 1) = 0101010101110101010101110101$$

$$D_1 = \text{Shift}(D_0, 1) = 1001100110011001100110011001$$

### Round 2: Left Shift by 1 Bit

$$C_2 = \text{Shift}(C_1, 1) = 1010101011101010101011101010$$

$$D_2 = \text{Shift}(D_1, 1) = 0011001100110011001100110011$$

## Round 3 Shift

### Round 3: Left Shift by 2 Bits

$$C_3 = \text{Shift}(C_2, 2) = 1010101110101010101110101010$$

$$D_3 = \text{Shift}(D_2, 2) = 1100110011001100110011001100$$

- Each subsequent round applies the shift schedule to ensure different subkeys.
- Shifting is circular, so bits that "fall off" the left end reappear on the right end.

# Overview of Compression P-Box

- The Compression P-Box is used in the key scheduling process of DES.
- It reduces the 56-bit combined  $C_n$  and  $D_n$  halves into a 48-bit subkey for each round.
- Permutation ensures that subkeys are not simple subsets of the original key.
- Operates by selecting and permuting 48 bits from the 56-bit key.

## Compression P-Box Table

The Compression P-Box permutes and selects bits based on the following table:

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 14 | 17 | 11 | 24 | 1  | 5  | 3  | 28 |
| 15 | 6  | 21 | 10 | 23 | 19 | 12 | 4  |
| 26 | 8  | 16 | 7  | 27 | 20 | 13 | 2  |
| 41 | 52 | 31 | 37 | 47 | 55 | 30 | 40 |
| 51 | 45 | 33 | 48 | 44 | 49 | 39 | 56 |
| 34 | 53 | 46 | 42 | 50 | 36 | 29 | 32 |

- This table defines how the 56-bit input is reduced to 48 bits.

## Example of Compression P-Box

**Input:**  $C_1 || D_1$  (56-bit key):

10101010101110101010101110101011101010101010101110101010

**Output:** Compressed 48-bit subkey:

111000011001101010101011110011011100110111001101

- The bits are selected and permuted based on the P-Box table.
- Example:
  - Bit 14 from the input becomes Bit 1 in the output.
  - Bit 17 from the input becomes Bit 2 in the output.
  - Continue this for all 48 positions.

## Detailed Example: Step-by-Step

- Given the first few bits of  $C_1 || D_1$ :

101010101011101010101011101010111010101010101011

- Compression P-Box mapping:

Input Bit 14 (1)  $\rightarrow$  Output Bit 1 (1)

Input Bit 17 (0)  $\rightarrow$  Output Bit 2 (0)

Input Bit 11 (1)  $\rightarrow$  Output Bit 3 (1)

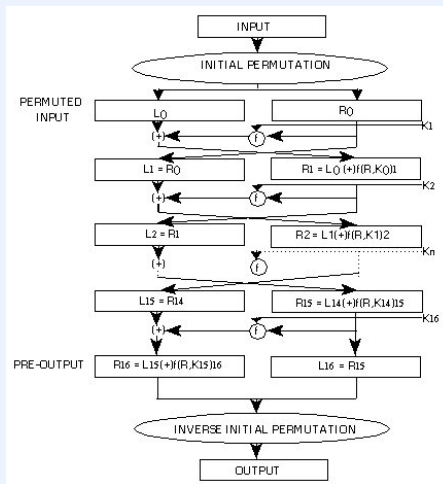
...  $\rightarrow$  ...

- Repeat this for all 48 bits to generate the subkey.

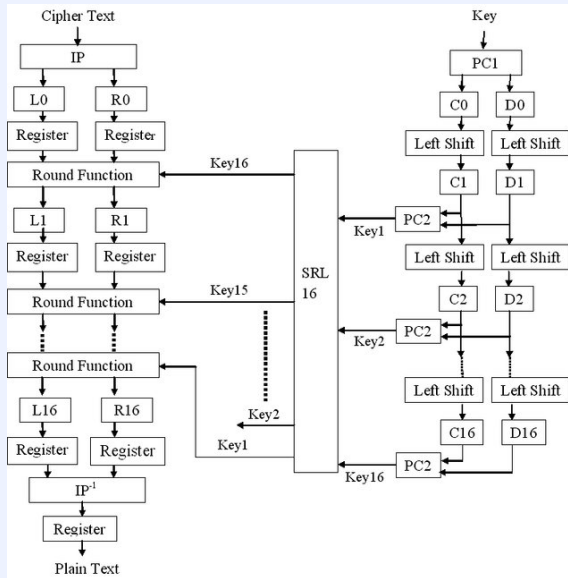


# DES Encryption Process

- ① **Initial Permutation (IP):** The input 64-bit block is permuted.
- ② **Split the Block:** Divide into two halves (Left and Right).
- ③ **16 Rounds of Feistel Structure:**
  - Expand the right half to 48 bits (Expansion function).
  - XOR with the round key.
  - Pass through S-boxes for substitution (reduce to 32 bits).
  - Permute the output and XOR with the left half.
- ④ **Swap Halves and Combine.**
- ⑤ **Final Permutation (inverse of IP).**



# DES Structure



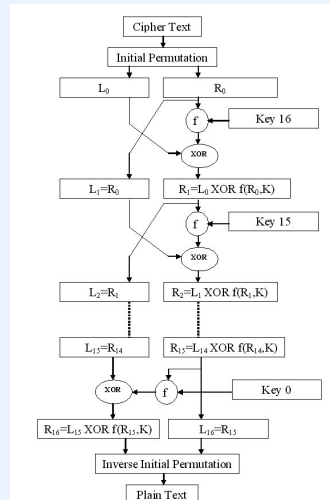
# DES Decryption Process

## • Identical Process to Encryption

- Decryption mirrors encryption but applies subkeys in reverse order.
- Ensures efficient implementation using the Feistel structure.

## • Final Step

- Reverse the final permutation to retrieve the original plaintext.



# Overview of Initial Permutation

- The Initial Permutation (IP) is the first step in the DES encryption process.
- It rearranges the 64-bit plaintext block based on a predefined permutation table.
- Ensures that the bits are shuffled for increased diffusion in subsequent rounds.

## Initial Permutation Table:

|    |    |    |    |    |    |    |   |
|----|----|----|----|----|----|----|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57 | 49 | 41 | 33 | 25 | 17 | 9  | 1 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 7 |

## Example of Initial Permutation

### Input 64-bit Block (Plaintext):

11010010 10011011 00101100 11101001 01010101 10101010 11001100 0

### Permutation Process:

- Rearrange the bits using the Initial Permutation Table.
- Example:
  - Bit 58 of the input becomes Bit 1 in the output.
  - Bit 50 of the input becomes Bit 2 in the output.
  - Continue this for all 64 bits.

### Output 64-bit Block (Permuted Plaintext):

01101100 11100010 11011010 11110011 10101001 00100100 01010101 1

# Introduction to Feistel Network

- A Feistel network is a symmetric structure used in the design of block ciphers.
- Named after Horst Feistel, it is widely used in cryptographic algorithms like DES.
- Key features:
  - Divides input block into two halves (left and right).
  - Performs multiple rounds of encryption using a round function and a key.
  - Symmetric: The same structure is used for decryption.

# Structure of Feistel Network

- Steps in one round:
  - ① Divide input into two halves:  $L_0$  and  $R_0$ .
  - ② Apply a round function  $F$  to the right half ( $R_0$ ) and combine with the left half ( $L_0$ ) using XOR.
  - ③ Swap the halves.
- Final round may skip the swap step.
- Given an input block  $P$ :
  - ① Divide  $P$  into two halves:  $L_0$  and  $R_0$ .
  - ② For each round  $i$ , compute:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1},$$

- ③ Repeat for  $n$  rounds.
- ④ Combine  $L_n$  and  $R_n$  to produce the ciphertext.

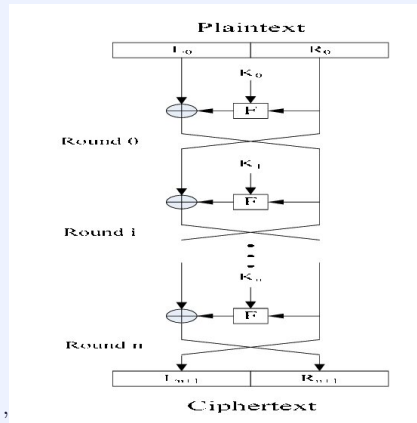
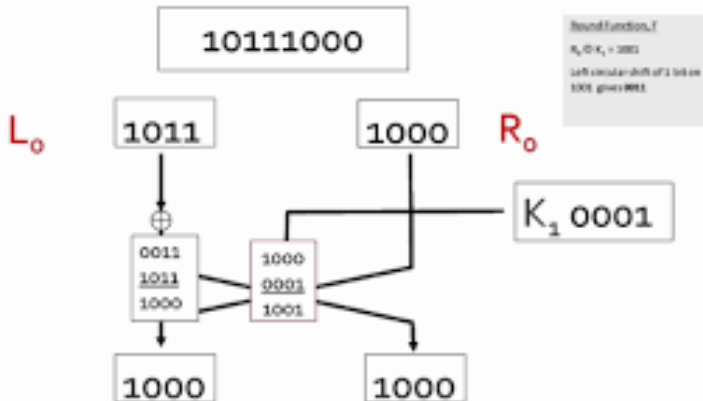


Figure: Basic Feistel Structure

# Fiestel Network Example





# Advantages of Feistel Network

- Symmetric structure allows encryption and decryption to use the same operations.
- High flexibility in designing the round function  $F$ .
- Proven security: Many modern block ciphers use variations of Feistel networks.
- Allows parallel processing of rounds for efficiency.

# Applications of Feistel Network

- Used in block ciphers like:
  - Data Encryption Standard (DES)
  - Triple DES (3DES)
  - Blowfish
- Basis for secure communication protocols.
- Applied in legacy systems requiring encryption and decryption.

## Example of Feistel Structure

### Input Block (64-bit Plaintext):

11001010 11101010 10111001 11001011 10101010 01010101 11001100 11110000

### Step 1: Divide into Halves

$L_0 = 11001010\ 11101010\ 10111001\ 11001011$ ,  $R_0 = 10101010\ 01010101\ 11001100\ 11110000$

### Step 2: Round Function and XOR

- ① Apply  $F(R_0, K_1)$ , where  $K_1$  is the first round key.
- ② Example  $F(R_0, K_1)$  output:

01100011 00110100 10101010 11101101

- ③ Compute:

$$L_1 = R_0$$

- ④ Compute:

$$R_1 = L_0 \oplus F(R_0, K_1)$$

- ⑤ Perform the XOR operation:

## Expansion (E-box) Table

**E-box Table (Expands 32 bits to 48 bits):**

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 32 | 1  | 2  | 3  | 4  | 5  |
| 4  | 5  | 6  | 7  | 8  | 9  |
| 8  | 9  | 10 | 11 | 12 | 13 |
| 12 | 13 | 14 | 15 | 16 | 17 |
| 16 | 17 | 18 | 19 | 20 | 21 |
| 20 | 21 | 22 | 23 | 24 | 25 |
| 24 | 25 | 26 | 27 | 28 | 29 |
| 28 | 29 | 30 | 31 | 32 | 1  |

Each row represents 6 expanded bits corresponding to the input 4 bits.

# Substitution (S-box) Table

## S-box 1:

| <i>Row/Col</i> | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7 |
|----------------|----|----|----|----|----|----|----|---|
| 0              | 14 | 4  | 13 | 1  | 2  | 15 | 11 | 8 |
| 1              | 3  | 10 | 6  | 12 | 5  | 9  | 0  | 7 |
| 2              | 0  | 15 | 7  | 4  | 14 | 2  | 13 | 1 |
| 3              | 10 | 6  | 12 | 11 | 9  | 5  | 3  | 8 |

Similar tables exist for S-boxes 2 through 8, each mapping 6 bits to 4 bits.

## Detailed Example: Feistel Function in DES

### Inputs:

- $R_0 = 10101010\ 01010101\ 11001100\ 11110000$  (32 bits)
- $K_1 =$   
 $01101101\ 10101011\ 11000011\ 10101011\ 10101001\ 11001100$   
 (48 bits)

### Step 1: Expansion (E-box)

- Expand  $R_0$  to 48 bits:

$$R_0^{\text{expanded}} = 11010101\ 01010110\ 10100101\ 11100111\ 00001111\ 1000$$

### Step 2: XOR with Round Key

$$F_{\text{XOR}} = R_0^{\text{expanded}} \oplus K_1$$

$$F_{\text{XOR}} = 10111000\ 11111101\ 01100110\ 01001100\ 10100110\ 01000110$$

## Detailed Example: Feistel Function in DES (cont.)

### Step 3: S-box Substitution

- Divide  $F_{\text{XOR}}$  into 8 groups of 6 bits.
- Example for the first group 101110:
  - Row: 10 (first and last bits) = 2 (decimal)
  - Column: 0111 (middle four bits) = 7 (decimal)
  - Lookup in S-box 1: 1110 (4 bits)
- Repeat for all groups to get a 32-bit output:

$$F_{\text{S-box}} = 11001010 \ 10110111 \ 00100110 \ 01110101$$

## Detailed Example: Feistel Function in DES (cont.)

### Step 4: Permutation (P-box)

- Apply the P-box permutation to  $F_{S\text{-box}}$ :

$$F_{P\text{-box}} = 01100011 \ 00110100 \ 10101010 \ 11101101$$

### Output of $F(R_0, K_1)$ :

$$F(R_0, K_1) = 01100011 \ 00110100 \ 10101010 \ 11101101$$

### P-box Table (32-bit permutation):

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 16 | 7  | 20 | 21 | 29 | 12 | 28 | 17 |
| 1  | 15 | 23 | 26 | 5  | 18 | 31 | 10 |
| 2  | 8  | 24 | 14 | 32 | 27 | 3  | 9  |
| 19 | 13 | 30 | 6  | 22 | 11 | 4  | 25 |

Each number indicates the new position of the corresponding bit after permutation.



## Summary of Feistel Function Example

- $R_0^{\text{expanded}} =$   
11010101 01010110 10100101 11100111 00001111 10001010
- $F_{\text{XOR}} =$   
10111000 11111101 01100110 01001100 10100110 01000110
- $F_{\text{S-box}} =$  11001010 10110111 00100110 01110101
- $F_{\text{P-box}} =$  01100011 00110100 10101010 11101101

This 32-bit output is XORed with  $L_0$  in the Feistel structure to compute the new halves.

# Strengths and Weaknesses of DES

## Strengths

- Simple and efficient design for hardware implementation.
- Use of multiple rounds increases cryptographic strength.
- The Feistel structure ensures invertibility.

## Weaknesses

- Limited key size (56 bits) makes it vulnerable to brute-force attacks.
- No longer secure for modern applications due to advancements in computational power.
- Susceptible to differential and linear cryptanalysis.

# Applications of DES

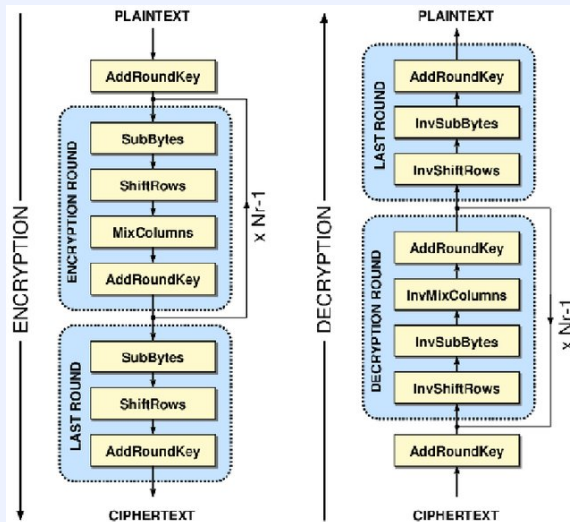
- Historically used in secure communications, financial transactions, and ATM PIN encryption.
- Forms the basis for more advanced algorithms like Triple DES (3DES).

# Block Ciphers: AES (Advanced Encryption Standard)

- Symmetric encryption algorithm standardized by NIST.
- Operates on 128-bit blocks using key sizes of 128, 192, or 256 bits.
- Highly secure and widely used in applications like SSL/TLS.



# AES Structure



# Overview of AES Rounds

- AES involves multiple rounds of encryption depending on the key size:
  - 10 rounds for 128-bit keys.
  - 12 rounds for 192-bit keys.
  - 14 rounds for 256-bit keys.
- Each round consists of:
  - 1 Substitute Bytes (SubBytes)
  - 2 Shift Rows
  - 3 Mix Columns
  - 4 Add Round Key
- Example here illustrates the first three steps in a single round.

## Step 1: Substitute Bytes (SubBytes)

- Each byte in the state matrix is substituted using an **S-box**, which is a predefined lookup table.
- Example:**

$$\text{Input State Matrix: } \begin{bmatrix} 0x19 & 0x3D & 0xE3 & 0xBE \\ 0xA0 & 0xF4 & 0xE2 & 0x2B \\ 0x9A & 0xC6 & 0x8D & 0x2A \\ 0xE9 & 0xF8 & 0x48 & 0x08 \end{bmatrix}$$

- S-box Substitution:** Each byte is replaced by its corresponding value in the S-box. For instance:

Substitute  $0x19 \rightarrow 0xD4$ ,  $0x3D \rightarrow 0x27$ ,  $0xE3 \rightarrow 0x11$

- Output State Matrix (after substitution):**

$$\begin{bmatrix} 0xD4 & 0x27 & 0x11 & 0xAE \\ 0x88 & 0x56 & 0x53 & 0x45 \\ 0x06 & 0x32 & 0xAC & 0x62 \\ 0x7C & 0xB5 & 0x41 & 0xC8 \end{bmatrix}$$



## Step 2: Shift Rows

- Rows in the state matrix are cyclically shifted.
- Example:

Input Matrix:

$$\begin{bmatrix} D4 & 27 & 11 & AE \\ 88 & 56 & 53 & 45 \\ 06 & 32 & AC & 62 \\ 7C & B5 & 41 & C8 \end{bmatrix}$$

- Shift Rows Operation:
  - Row 0: No shift.
  - Row 1: Left shift by 1.
  - Row 2: Left shift by 2.
  - Row 3: Left shift by 3.
- Result:

Output Matrix:

$$\begin{bmatrix} D4 & 27 & 11 & AE \\ 56 & 53 & 45 & 88 \\ AC & 62 & 06 & 32 \\ C8 & 7C & B5 & 41 \end{bmatrix}$$

## Step 3: Mix Columns

- Each column is multiplied by a fixed matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

- Example:

Input Matrix:

$$\begin{bmatrix} D4 & 27 & 11 & AE \\ 56 & 53 & 45 & 88 \\ AC & 62 & 06 & 32 \\ C8 & 7C & B5 & 41 \end{bmatrix}$$

- Result after MixColumns:

Output Matrix:

$$\begin{bmatrix} 04 & 66 & 81 & E5 \\ E0 & CB & 19 & 9A \\ 7A & 0E & F1 & 77 \\ 3D & 5D & E0 & 88 \end{bmatrix}$$

# Advantages of Symmetric Key Cipher

- Faster than asymmetric key encryption.
- Simple to implement with lower computational requirements.
- Efficient for encrypting large amounts of data.

# Challenges of Symmetric Key Cipher

- Key distribution: Securely sharing the secret key is challenging.
- Scalability: Requires separate keys for each pair of users.
- Vulnerability: If the key is compromised, all communications are exposed.

# Applications of Symmetric Key Cipher

- Data Encryption: Secure storage of sensitive data.
- Network Security: Protecting communication channels (e.g., SSL/TLS, VPNs).
- Secure File Transfers: Encrypting files during transmission.

# Symmetric vs. Asymmetric Encryption

| Feature   | Symmetric                  | Asymmetric             |
|-----------|----------------------------|------------------------|
| Key Usage | Single key                 | Public and private key |
| Speed     | Faster                     | Slower                 |
| Security  | Key sharing is a challenge | More secure            |
| Examples  | AES, DES                   | RSA, ECC               |

# Introduction to Asymmetric Key Ciphers

- Uses two separate keys: Public Key (used for encryption) and Private Key (used for decryption).
- Ensures confidentiality, authenticity, and non-repudiation.
- Eliminates the need to securely exchange a shared key.
- Examples: RSA, ECC, DSA.

# How It Works

- Public key is shared openly, while the private key is kept secret.
- Encryption:
  - Sender encrypts the message using the recipient's public key.
- Decryption:
  - Recipient decrypts the message using their private key.



# RSA Algorithm

- Based on the mathematical difficulty of factoring large prime numbers.
- Steps:
  - ① Key generation: Generate public and private keys.
  - ② Encryption:  $C = M^e \bmod n$ .
  - ③ Decryption:  $M = C^d \bmod n$ .
- Widely used in secure communications and digital signatures.

## Step 1: Key Generation

- Choose two prime numbers  $p = 61$  and  $q = 53$ .
- Compute  $n = p \cdot q = 61 \cdot 53 = 3233$ .
- Compute  

$$\phi(n) = (p - 1)(q - 1) = (61 - 1)(53 - 1) = 60 \cdot 52 = 3120.$$
- Choose  $e$  such that  $1 < e < \phi(n)$  and  $\gcd(e, \phi(n)) = 1$ .

$$e = 17$$

- Compute  $d$ , the modular multiplicative inverse of  $e \bmod \phi(n)$ :

$$d \cdot e \equiv 1 \pmod{\phi(n)} \Rightarrow d = 2753$$

- Public Key:  $(e, n) = (17, 3233)$
- Private Key:  $(d, n) = (2753, 3233)$

## Step 2: Encryption

- Message to encrypt:  $M = 65$ .
- Use the public key  $(e, n)$  to compute ciphertext  $C$ :

$$C = M^e \mod n = 65^{17} \mod 3233$$

- Simplify using modular arithmetic:

$$C = 2790$$

- Encrypted message:  $C = 2790$

# Problem Statement

- Find the multiplicative inverse of  $a = 3$  modulo  $m = 26$ .
- This means finding  $x$  such that:

$$3x \equiv 1 \pmod{26}$$

- Use the Extended Euclidean Algorithm to solve:

$$3x + 26y = 1$$

# Extended Euclidean Algorithm

| <b>a</b> | <b>b</b> | <b>q</b> | <b>r</b> | <b>s1</b> | <b>s2</b> | <b>s3</b> | <b>t1</b> | <b>t2</b> | <b>t3</b> |
|----------|----------|----------|----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 161      | 28       | 5        | 21       | 1         | 0         | 1         | 0         | 1         | -5        |
| 28       | 21       | 1        | 7        | 0         | 1         | -1        | 1         | -5        | 6         |
| 21       | 7        | 3        | 0        | 1         | -1        | 4         | -5        | 6         | -23       |

Table: Sample Data Table

# Elliptic Curve Cryptography (ECC)

- Uses properties of elliptic curves over finite fields for security.
- Provides same security level as RSA with smaller key sizes.
- Steps:
  - ① Key generation based on point multiplication.
  - ② Encryption using shared public and private keys.
  - ③ Decryption reverses the elliptic curve operations.
- Commonly used in IoT and mobile devices.

# Applications and Advantages

- Applications:
  - Secure email communication (PGP/GPG).
  - Digital signatures for authentication.
  - SSL/TLS protocols for secure web connections.
  - Cryptocurrencies and blockchain technology.
- Advantages:
  - No need to securely exchange keys.
  - High scalability for large networks.
  - Stronger security with smaller keys (e.g., ECC).